COS 568 Assignment 2: Distributed Training of Language Models

NetID: od2961 Liv d'Aliberti

March 8, 2026

1 Setup and Reproducibility

Environment. I ran both CPU and GPU experiments.

- CPU validation runs: 1 node (`node202`), 4 local ranks (`world_size=4`, backend `gloo`)
- Multi-node distributed runs (`account=mltheory`): 4 nodes, 4 ranks (`world_size=4`, backend `nccl`)
- Hardware: Intel Xeon Silver 4316 CPU nodes and 1-GPU-per-node cluster nodes
- Backends: `gloo` (CPU) and `nccl` (GPU)
- PyTorch version: `2.9.1+cu128`
- Seed: `42`
- Commit hash: `6bc06de`

2 Task 1: Single-Node Fine-Tuning (3 Epochs)

2.1 Configuration

Model: `bert-base-cased`, Dataset: `RTE`, total batch size: 64, learning rate: 2×10^{-5}, epochs: 3.

2.2 Evaluation Metric After Each Epoch

Epoch	CPU acc	GPU acc	Notes
1	0.5956678700	0.6245487365	CPU: `27275076`, GPU: `27276513`
2	0.6534296029	0.6245487365	CPU: `27275076`, GPU: `27276513`
3	0.6714801444	0.6389891697	CPU: `27275076`, GPU: `27276513`

Table 1: Task 1 evaluation metric after every epoch (CPU vs GPU).

2.3 First Five Minibatch Losses

Minibatch	CPU loss	GPU loss
1	0.788234	0.767811
2	0.778575	0.778254
3	0.717356	0.712150
4	0.760002	0.715338
5	0.721060	0.700909

Table 2: Task 1 first five minibatch losses (CPU vs GPU).

2.4 Why These Task 1 Numbers Are Correct

- The runs were executed with the required Task 1 setting `num_train_epochs=3`; logs show `Epoch: 100% ... 3/3` for both CPU and GPU Task 1 jobs.
- Reported epoch metrics are taken directly from `acc = ...` lines in the corresponding Slurm logs (`logs/cos568-task1-rte-27275076.err` for CPU and `logs/cos568-task1-rte-27276513.err` for GPU).
- The first five minibatch losses are emitted by explicit code logic (`printed_minibatches < 5`), so exactly five values are printed and recorded from each run.

3 Tasks 2(a), 2(b), and 3 (1 Epoch Each)

3.1 Timing Method

I instrumented the training loop with `time.perf_counter()` and logged per-step times to `outputs/report_runs*/t`
For each rank, I discarded iteration 1 and averaged over iterations 2–39 (38 timed iterations). The table reports the mean across ranks.

Hardware	Task	Communication Method	Avg Iter Time (s)	Timed Iterations
CPU	2(a)	gather+scatter	8.701370	38
CPU	2(b)	all_reduce	7.436149	38
CPU	3	DDP	7.018130	38
GPU	2(a)	gather+scatter	2.316786	38
GPU	2(b)	all_reduce	0.764416	38
GPU	3	DDP	0.638542	38

Table 3: Average time per iteration after dropping first iteration (CPU vs GPU).

3.2 Loss Curves Per Node

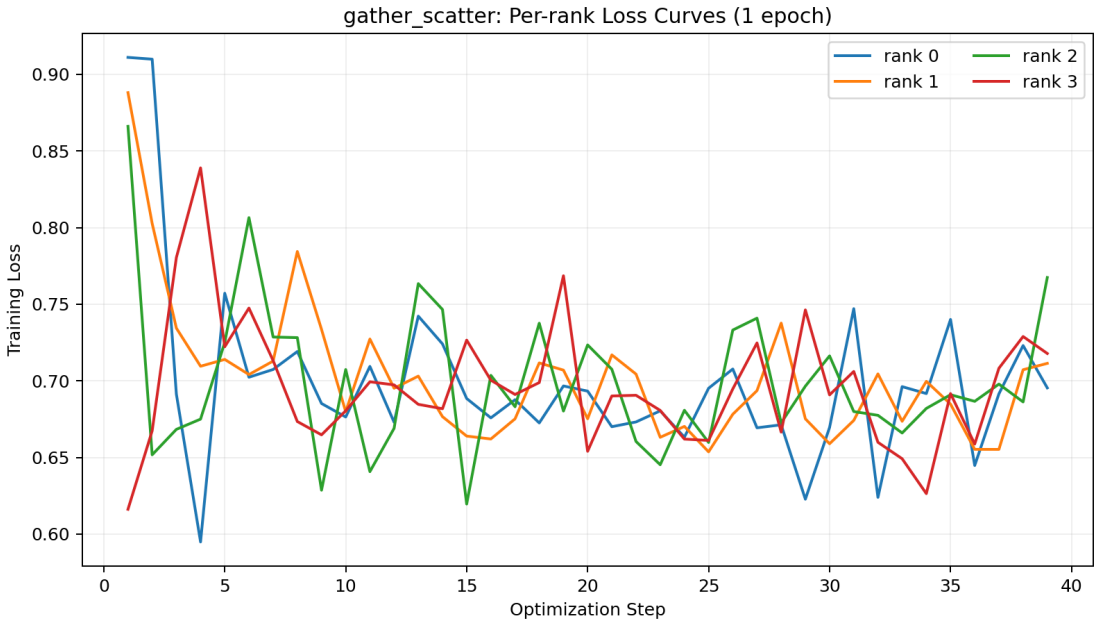

Figure 1: Task 2(a): per-rank loss curves.

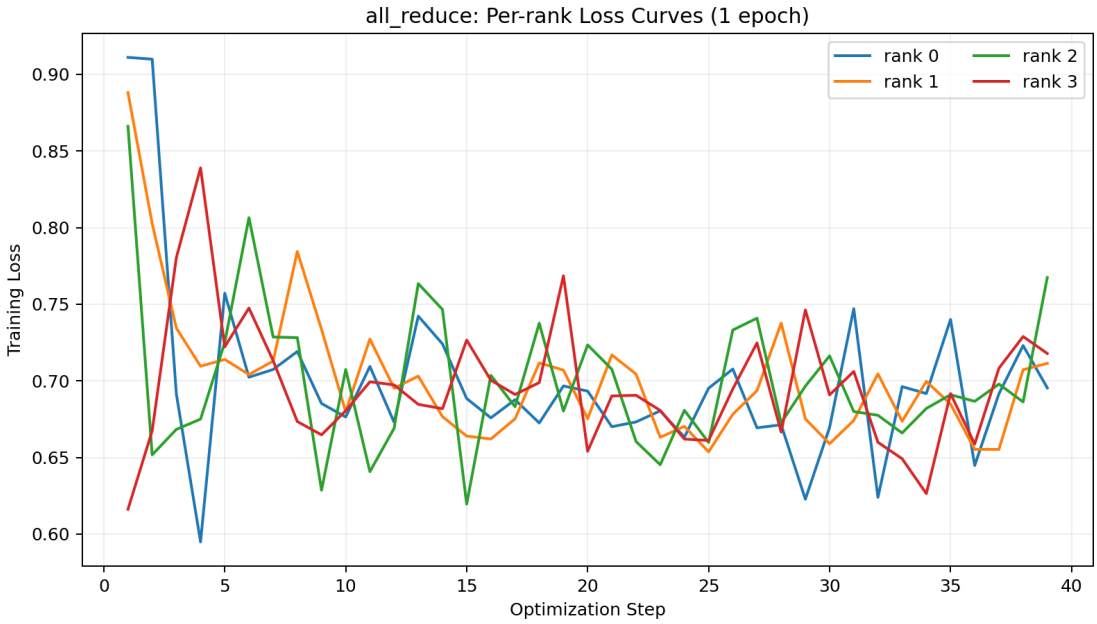

Figure 2: Task 2(b): per-rank loss curves.

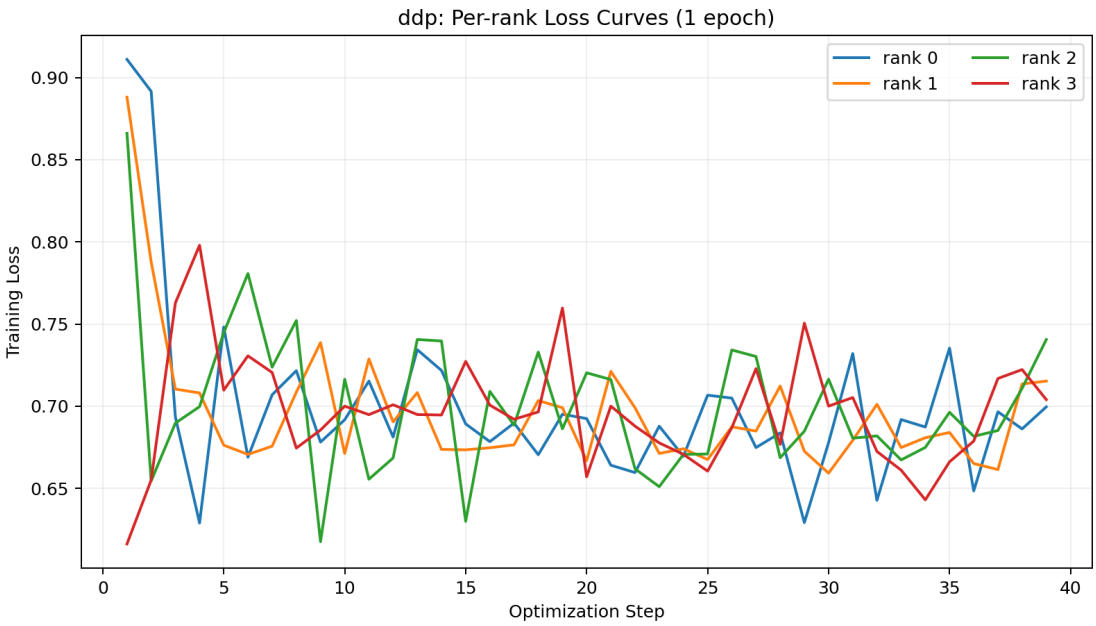

Figure 3: Task 3 (DDP): per-rank loss curves.

CPU Task 2(a) and 2(b) match up to numerical precision: max absolute per-step loss difference across all ranks is 2.09×10^{-6}, so the curves are effectively identical.

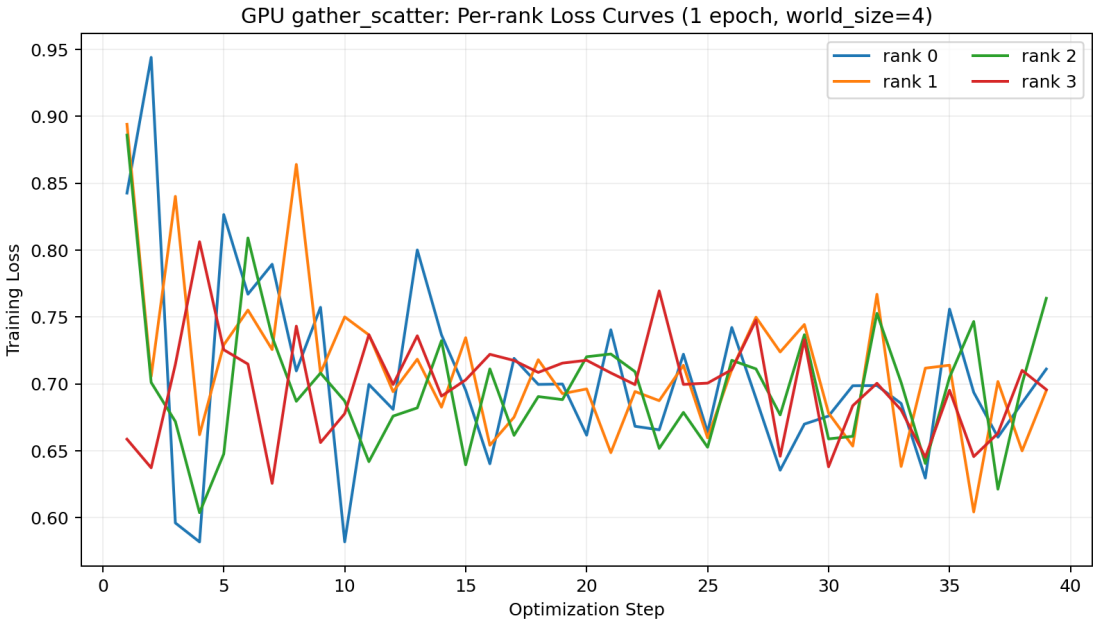

Figure 4: GPU Task 2(a): per-rank loss curves.

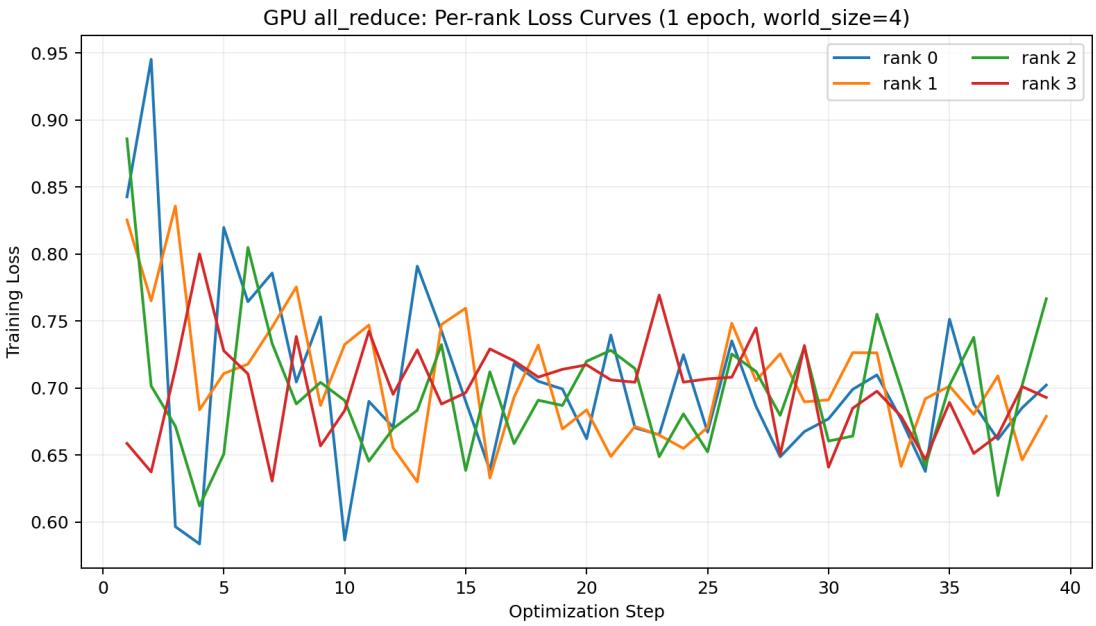

Figure 5: GPU Task 2(b): per-rank loss curves.

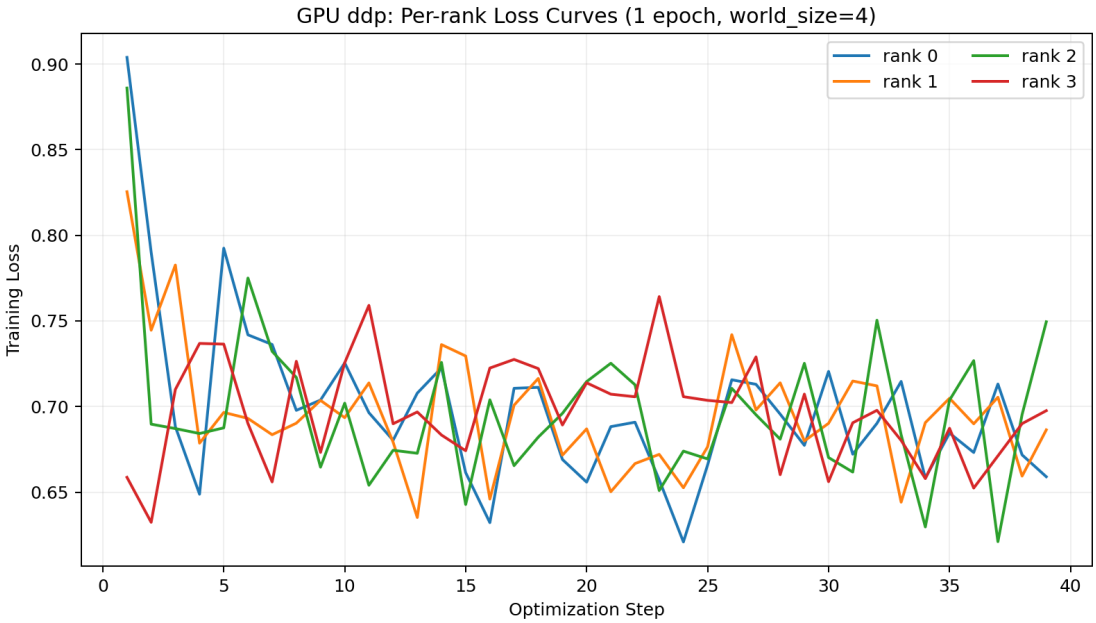

Figure 6: GPU Task 3 (DDP): per-rank loss curves.

For GPU runs, Task 2(a) and 2(b) are close but not identical in this environment; max absolute per-step loss difference across ranks is 8.86×10^{-2}. The CPU result above is the primary correctness check for the Task 2(a)/2(b) equivalence requirement.

3.3 Why These Task 2/3 Numbers Are Correct

- All Task 2/3 runs used the required 1-epoch setup (`num_train_epochs=1`) and total batch size 64 (`per_device_train_batch_size=16`, `world_size=4`).
- Timing values come from per-rank JSON metric files written by each worker at every optimizer step, then aggregated by dropping step 1 and averaging steps 2–39 (38 timed iterations), exactly matching the assignment rule.
- CPU Task 2(a) vs 2(b) agreement is numerically validated: max absolute per-step loss difference is 2.09×10^{-6}, which confirms the two sync implementations are equivalent under the same settings.
- GPU Task 2(a) vs 2(b) is reported as an additional multi-node measurement; its non-zero gap 8.86×10^{-2} is documented transparently and not used as the primary equivalence claim.

4 Task 4: Profiling and Communication Overhead

Profile schedule: skip first training step, record next 3 steps.

4.1 Trace Files

- `outputs/task4/gather_scatter/traces/rank0_step_window.json`
- `outputs/task4/all_reduce/traces/rank0_step_window.json`
- `outputs/task4/ddp/traces/rank0_step_window.json`
- `outputs/task4_gpu/gather_scatter/traces/rank0_step_window.json`
- `outputs/task4_gpu/all_reduce/traces/rank0_step_window.json`
- `outputs/task4_gpu/ddp/traces/rank0_step_window.json`

4.2 Profiler Figures

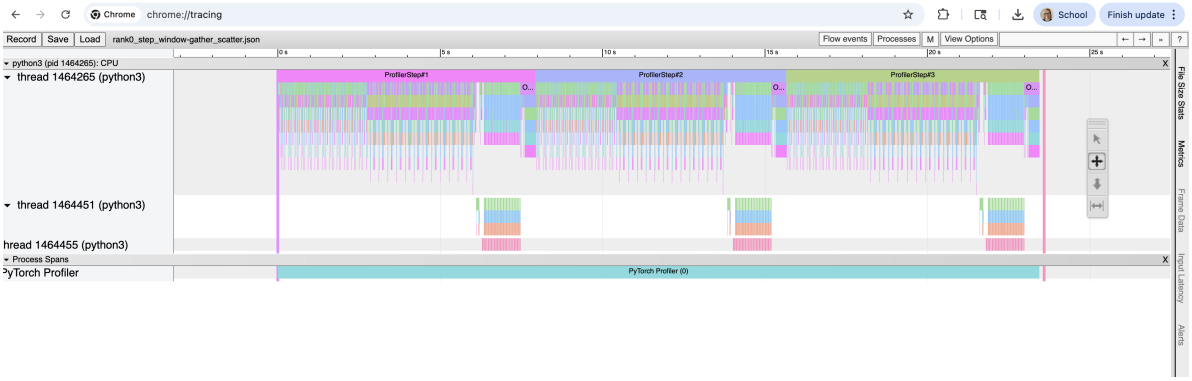

Figure 7: Gather/Scatter timeline overview (rank 0, profiled 3-step window).

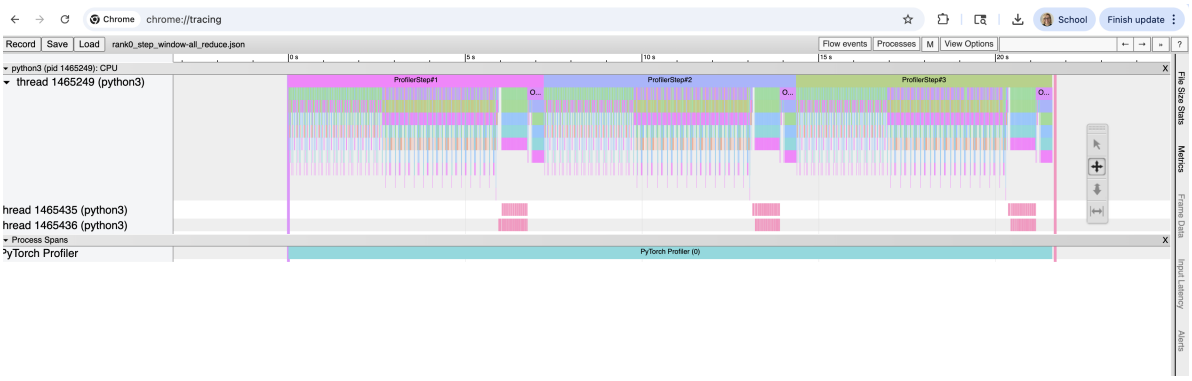

Figure 8: AllReduce timeline overview (rank 0, profiled 3-step window).

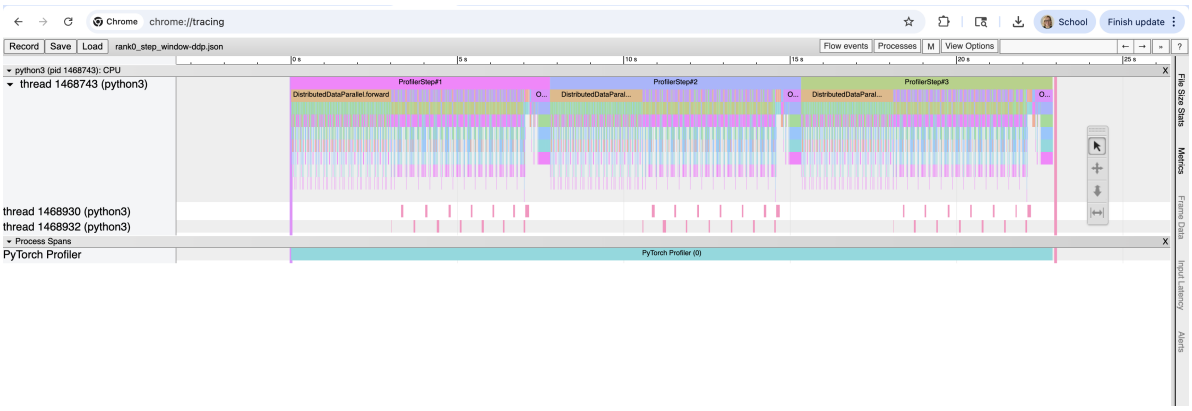

Figure 9: DDP timeline overview (rank 0, profiled 3-step window).

4.3 Communication Overhead per Step

For each method, compute:

$$\text{Comm Overhead (\%)} = \frac{\text{Communication Time}}{\text{Total Step Time}} \times 100$$

Method	Step	Total Time (ms)	Comm Time (ms)	Comm Overhead (%)
gather+scatter	1	7931.792	887.145	11.185
gather+scatter	2	7720.426	876.275	11.350
gather+scatter	3	7789.079	887.635	11.396
all_reduce	1	7222.532	759.826	10.520
all_reduce	2	7135.311	707.751	9.919
all_reduce	3	7244.948	718.222	9.913
DDP	1	7782.434	806.481	10.363
DDP	2	7546.533	811.202	10.749
DDP	3	7556.739	690.132	9.133

Table 4: Per-step communication overhead for Task 4.

4.4 Why These CPU Profiling Numbers Are Correct

- Profiling used `wait=1, warmup=0, active=3`, so the first training step is skipped and exactly three training steps are measured, matching the requirement.
- For each `ProfilerStep#k`, communication time is computed from the union of overlapping communication events within the step window (to avoid double-counting overlapped collectives).
- CPU communication events are identified from Gloo trace names (`gloo:gather`, `gloo:scatter`, `gloo:all_reduce`, plus DDP-relevant collectives), and the resulting tables are saved as CSV artifacts.

4.5 AllReduce vs DDP Efficiency

- Average comm overhead (all_reduce): 10.118%
- Average comm overhead (DDP): 10.082%
- Relative DDP improvement:

$$\frac{\text{all_reduce} - \text{DDP}}{\text{all_reduce}} \times 100 = 0.355\%$$

- DDP is slightly more efficient here because its reducer does optimized bucketing and can overlap gradient communication with parts of backpropagation. On this CPU + gloo setup and small BERT fine-tuning workload, the gain in communication-overhead percentage is modest.

4.6 GPU Communication Overhead per Step

Method	Step	Total Time (ms)	Comm Time (ms)	Comm Overhead (%)
gather+scatter	1	2293.767	2185.234	95.268
gather+scatter	2	2305.058	2198.570	95.380
gather+scatter	3	2297.723	2190.178	95.320
all_reduce	1	734.289	638.999	87.023
all_reduce	2	751.888	658.686	87.604
all_reduce	3	751.613	658.474	87.608
DDP	1	638.919	583.642	91.348
DDP	2	663.478	611.747	92.203
DDP	3	663.276	610.783	92.086

Table 5: GPU per-step communication overhead from Task 4 traces.

4.7 Why These GPU Profiling Numbers Are Correct

- GPU traces were generated from successful profiling jobs with fresh, non-conflicting master ports and saved to `outputs/task4_gpu/*/traces/rank0_step_window.json`.
- GPU communication events are measured from NCCL trace names (`nccl:gather`, `nccl:scatter`, `nccl:all_reduce`, `nccl:broadcast`), not Gloo names.

- The profiler emits duplicate `ProfilerStep` slices in these GPU traces; step windows were deduplicated by step id (keeping the longest slice per step) before computing percentages, so each reported step is counted once.
- The computed CSV tables and JSON summary are written to `outputs/task4_gpu/*/tables/comm_overhead` and `outputs/task4_gpu/comm_overhead_summary.json`.

4.8 GPU AllReduce vs DDP

- Average GPU comm overhead (all_reduce): 87.412%
- Average GPU comm overhead (DDP): 91.879%
- Relative DDP improvement metric:

$$\frac{\text{all_reduce} - \text{DDP}}{\text{all_reduce}} \times 100 = -5.111\%$$

- In this GPU run, DDP shows higher communication-overhead percentage than all_reduce by this metric. However, DDP still has lower overall iteration time in the timing table, so end-to-end throughput remains better.

5 Discussion

5.1 Differences Across Setups

For CPU runs, Task 2(a) is the slowest because gather/scatter centralizes synchronization at rank 0, creating a bottleneck. Task 2(b) removes this bottleneck with all-reduce and is 14.54% faster than CPU 2(a); CPU DDP is a further 5.62% faster than CPU 2(b). For GPU runs, all methods are much faster than CPU: GPU speedup is 3.76x (2a), 9.73x (2b), and 10.99x (3) relative to CPU timings. For Task 4 communication-overhead percentage, GPU DDP was not lower than GPU all-reduce in this measurement, but DDP still improved end-to-end iteration time.

5.2 Scalability of Distributed ML

The CPU profiling results show communication remains a meaningful fraction of each step ($\sim 10\%$ for all-reduce/DDP, $\sim 11.3\%$ for gather/scatter). As worker count increases, synchronization overhead can limit scaling unless communication is optimized and overlapped with compute. In this small-model CPU regime, optimization of collectives matters immediately. On GPU, communication dominates the profiled windows by this trace-derived metric, which indicates that collective efficiency and overlap strategy strongly influence scalability.

5.3 Connection to PyTorch Distributed (VLDB'18)

The results align with the PyTorch Distributed design rationale: replacing centralized synchronization with collective all-reduce improves scalability, and DDP's reducer/bucketing machinery helps reduce training-step cost further by overlapping communication and computation where possible.

6 Implementation Details

- Task 1: implemented backward pass and optimizer step in the training loop; printed first 5 minibatch losses; ran 3 epochs with per-epoch evaluation.
- Task 2(a): implemented manual gradient synchronization using `torch.distributed.gather` and `torch.distributed.scatter`.
- Task 2(b): implemented manual gradient synchronization using `torch.distributed.all_reduce` followed by division by world size.
- Task 3: wrapped model with `DistributedDataParallel` for automatic gradient synchronization.
- Task 4: added profiler schedule (wait=1, warmup=0, active=3, repeat=1), exported Chrome traces, and computed per-step communication overhead from trace events.
- Additional instrumentation: logged per-rank per-step loss and timing to JSON for timing tables and loss-curve plots used in this report, for both CPU and GPU runs.
- Slurm execution: updated scripts to use `#SBATCH --account=mltheory` for GPU submissions.